Lecture 08 - Dictionaries and Binary Search Trees

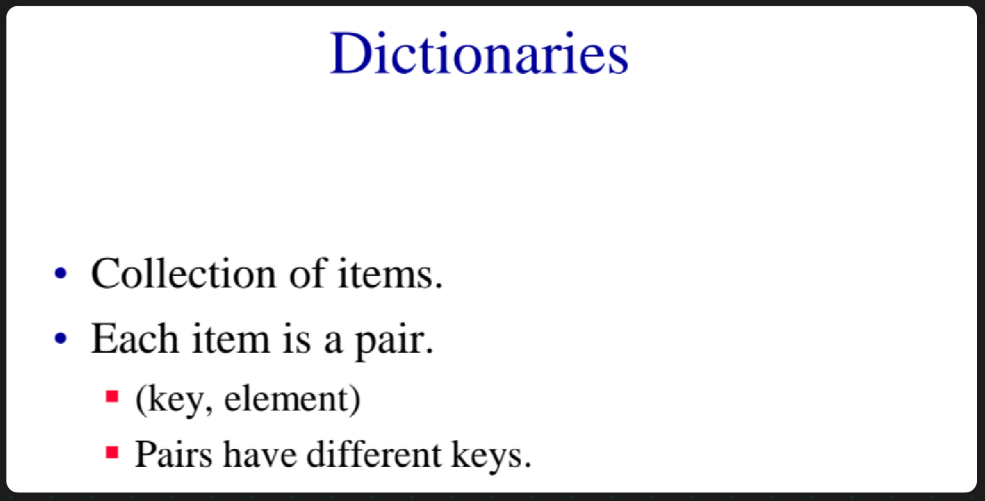
This lecture studies dictionaries that support more than exact-key lookup. Search trees can also find the closest qualifying key, visit keys in sorted order, and retrieve an item by its position in that order.

Lecture flow: dictionary operations -> best-fit bin packing -> indexed search trees -> optimal static search trees -> dynamic BST operations -> why balanced search trees are needed.

The two kinds of optimization in this lecture serve different purposes:

- An **optimal static BST** minimizes expected search cost for known access probabilities.
- A **balanced dynamic BST** keeps its height logarithmic as items are inserted and removed.

1. Dictionaries and the Operations We Need



Application

- Collection of student records in this class.
  - (key, element) = (student name, linear list of assignment and exam scores)
  - All keys are distinct.
- Collection of in-use domain names.
  - (godaddy.com, owner information)
  - All keys are distinct.

Dictionary With Duplicates

- Keys are not required to be distinct.
- Word dictionary.
  - Items/pairs are of the form (word, meaning).
  - May have two or more entries for the same word.
    - (bolt, a threaded pin)
    - (bolt, a crash of thunder)
    - (bolt, to shoot forth suddenly)
    - (bolt, a gulp)
    - (bolt, a standard roll of cloth)
    - etc.

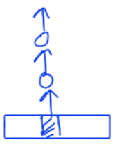
Dictionary Operations

- Static Dictionary.
  - initialize/create
  - get(theKey) (a.k.a. search)
  - CD ROM word dictionary
  - CD ROM geographic database of cities, rivers, roads, auto navigation system, etc.
- Dynamic Dictionary.
  - get(theKey) (a.k.a. search)
  - put(theKey, theElement) (a.k.a. insert)
  - remove(theKey) (a.k.a. delete)

we can't change after we initialize it.
(CD ROM 등록 후에)

Why a hash table is not always sufficient

Hash Table Dictionaries



- O(1) expected time for get, put, and remove.
- O(n) worst-case time for get, put, and remove.
  - O(log n) if overflows handled by balanced search trees.
- Not suitable for nearest match queries.
  - Get element with smallest key >= theKey.
- Not suitable for range queries.
- Not suitable for indexed operations.
  - Get element with third smallest key.
  - Remove element with 5th smallest key.





6/75

Hash tables offer expected $O(1)$ exact lookup, insertion, and deletion under suitable hashing and load-factor assumptions. A basic implementation can take $O(n)$ in the worst case because many keys may collide. Balanced search trees inside collision buckets can improve the bucket-search bound.

However, hash values do not preserve key order. A basic hash table does not directly support efficient:

- **Nearest-match queries:** find the smallest key greater than or equal to a given value.
- **Range queries:** find all keys between two bounds.
- **Indexed queries:** get or remove the item with the third-smallest key.
- **Sorted output:** visit every item in ascending key order.

A BST preserves the ordering rule:

$$\text{all left-subtree keys} < \text{node key} < \text{all right-subtree keys}$$

This makes ordered queries natural. Their efficiency depends on the tree's height.

2. Best-Fit Bin Packing with a Search Tree

Bin Packing

An example for "why we need nearest match"

- n items to be packed into bins
- each item has a size
- each bin has a capacity of c
- minimize number of bins

- Best Fit.
  - Items are packed one at a time in given order.
  - To determine the bin for an item, first determine set S of bins into which the item fits.
  - If S is empty, then start a new bin and put item into this new bin.
  - Otherwise, pack into bin of S that has least available capacity.

Best fit processes items in their given order. For an item of size s, choose the bin with the **smallest remaining capacity that is still at least** s. If no existing bin can fit the item, open a new bin. For capacity 10 and item sizes $[4, 7, 3, 6]$:

Item	Decision	Remaining capacities afterward
4	Open bin 1	Bin 1: 6
7	Open bin 2 because 7 does not fit in bin 1	Bin 1: 6, bin 2: 3
3	Choose bin 2, which fits exactly	Bin 1: 6, bin 2: 0
6	Choose bin 1, which fits exactly	Bin 1: 0, bin 2: 0

n = 4

weights = [4, 7, 3, 6]

capacity = 10

bins 4 ති තියෙනවා, එතන capacity එක 10 වගේ

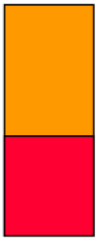


Pack red item into first bin.

n = 4

weights = [4, 7, 3, 6]

capacity = 10



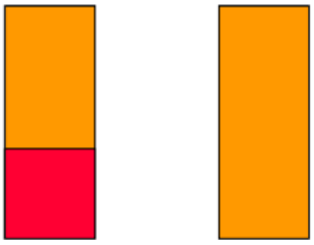
Pack blue item next.

Doesn't fit, so start a new bin.

n = 4

weights = [4, 7, 3, 6]

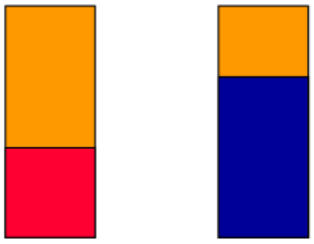
capacity = 10



n = 4

weights = [4, 7, 3, 6]

capacity = 10

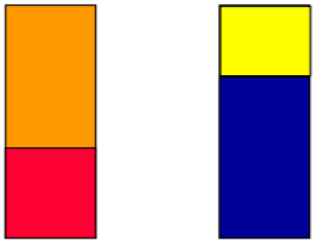


Pack yellow item into second bin.

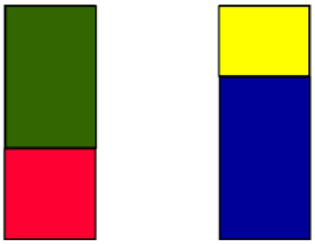
n = 4

weights = [4, 7, 3, 6]

capacity = 10



Pack green item into first bin.



Optimal packing.

This example is optimal because the total size is 20 and two capacity-10 bins are necessary. Best fit is a heuristic; *it is not guaranteed to find an optimal packing on every input.*

Finding the Nearest Qualifying Key

Implementation Of Best Fit

- Use a dynamic dictionary (with duplicates) in which the items are of the form (available capacity, bin index). initially empty
- Pack an item whose requirement is s.
  - Find a bin with smallest available capacity >= s. hash table fail good enough
  - Reduce available capacity of this bin by s.
    - May be done by removing old pair and inserting new one.
  - If no such bin, start a new bin.
    - Insert a new pair into the dictionary.

Store each bin in a dynamic dictionary as:

$$(\text{remaining capacity}, \text{bin index})$$

Duplicate capacities are allowed because different bins may have the same space remaining. An implementation may instead use a unique composite key `(remaining capacity, bin index)`.
Packing an item of size s requires a **lower-bound search**:

$$\min\{\text{remaining capacity} \mid \text{remaining capacity} \geq s\}$$

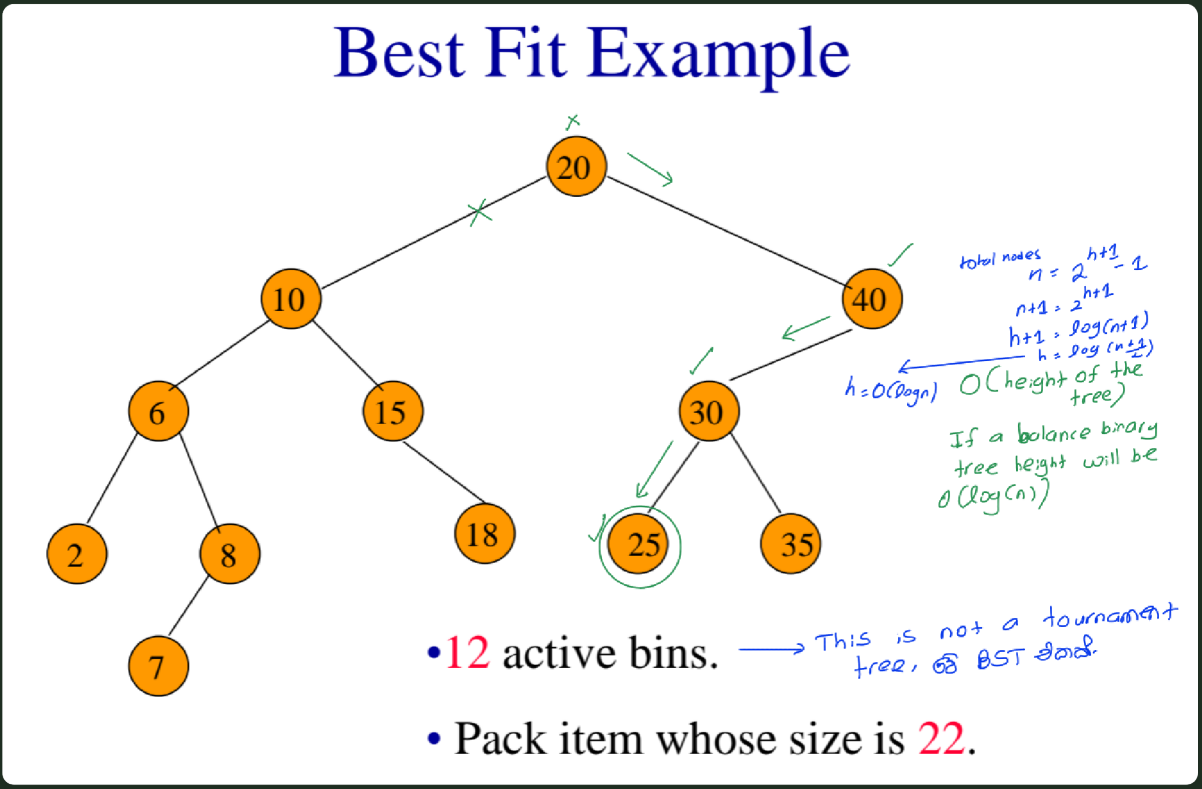
c++

```
findBestFit(root, s):
    candidate = null
    x = root

    while x is not null:
        if x.capacity >= s:
            candidate = x
```

```
            x = x.left        // try to find a smaller
    capacity that still fits
        else:
            x = x.right       // x and its left subtree
    are too small

    return candidate
```



For size 22, the search visits capacities:

```text
20 -> 40 -> 30 -> 25
```

20 is too small. Capacities 40, 30, and 25 are candidates, with 25 the best. After packing the item, the chosen bin has $25 - 22 = 3$ units remaining.

Remove its old dictionary entry and insert the updated entry. Simply changing 25 to 3 in its current position could violate BST order.

Complexity Of Best Fit



- Use a balanced binary search tree (with duplicates) in which the pairs are (available capacity, bin index).
- O(n) get, put, and remove/put operations, where n is the number of items to be packed.
- O(n log n).



Important

With a balanced BST, the search, deletion, and insertion each take $O(\log b)$ for b current bins. Since $b \leq n$, processing n items takes:

$$O(n \log n)$$

3. Indexed Binary Search Trees

Indexed Binary Search Tree

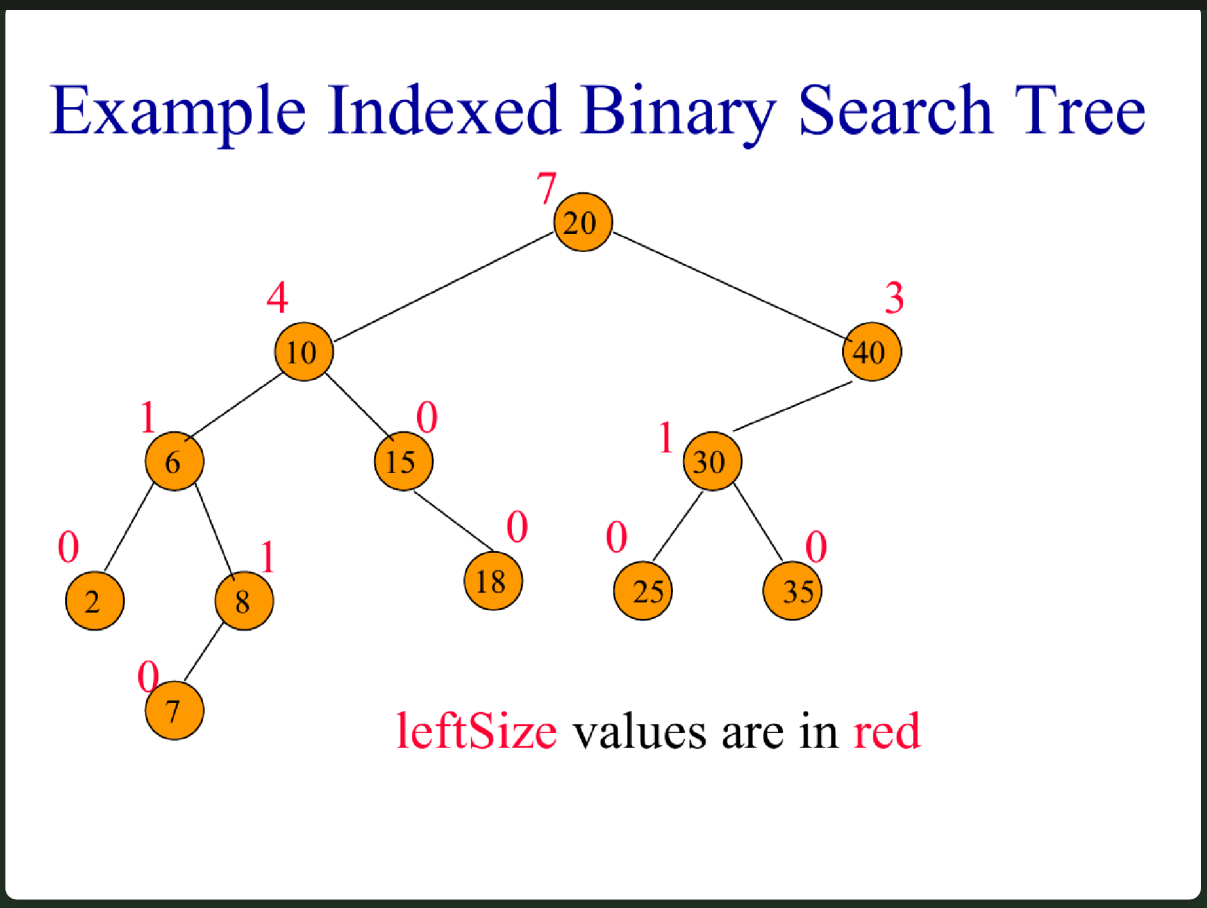
* Sometimes we want to find the index ⟶ e.x. want to find the 7th item in the list.

- Binary search tree.
- Each node has an additional field.
  - leftSize = number of nodes in its left subtree

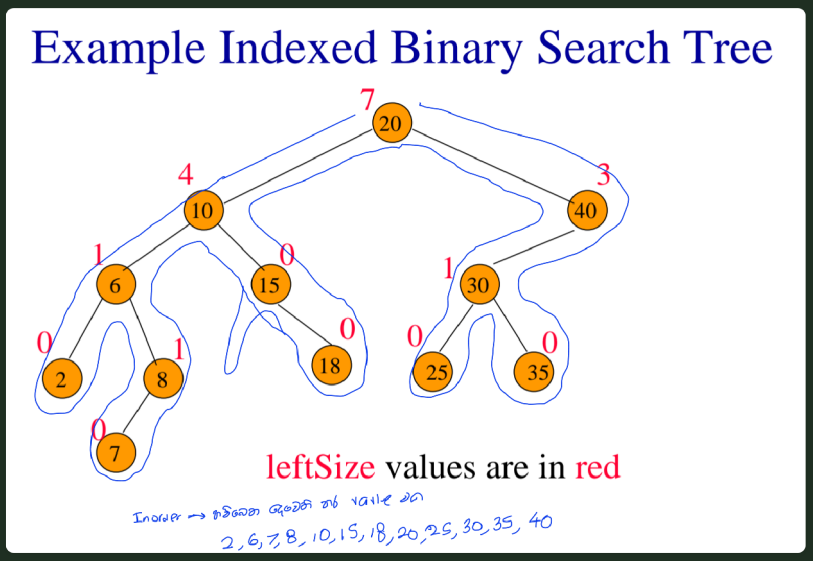
An **indexed BST** augments every node with:

$$\text{leftSize}(x) = \text{number of nodes in the left subtree of } x$$

The field tells us how many items come before that node within its own subtree.



> **Important**
> In order traversal gives the sorted sequence in BST:




```text
[2, 6, 7, 8, 10, 15, 18, 20, 25, 30, 35, 40]
```

The lecture uses **zero-based rank**:

$$\text{rank}(2) = 0, \qquad \text{rank}(15) = 5, \qquad \text{rank}(20) = 7$$

leftSize And Rank

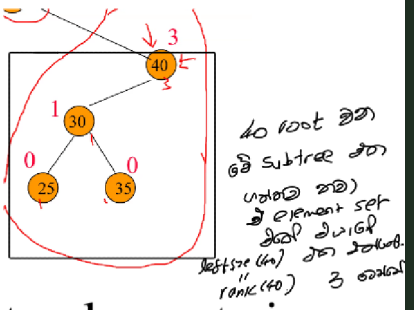
Rank of an element is its position in inorder (inorder = ascending key order).

[2,6,7,8,10,15,18,20,25,30,35,40]

rank(2) = 0

rank(15) = 5

rank(20) = 7



lextSize(x) = rank(x) with respect to elements in subtree rooted at x

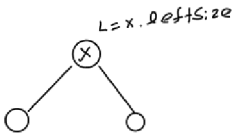
not whole tree.

`leftSize` is a local rank within a node's subtree. It is not necessarily that node's rank in the whole tree. For example, node 15 has `leftSize = 0` but global rank 5.

Retrieving an Item by Index

At node x, let $L = x.\text{leftSize}$ and let i be the desired index within the current subtree.

get(index)



- if index = x.leftSize desired element is x.element
- if index < x.leftSize desired element is index'th element in left subtree of x
- if index > x.leftSize desired element is (index – x.leftSize – 1)'th element in right subtree of x

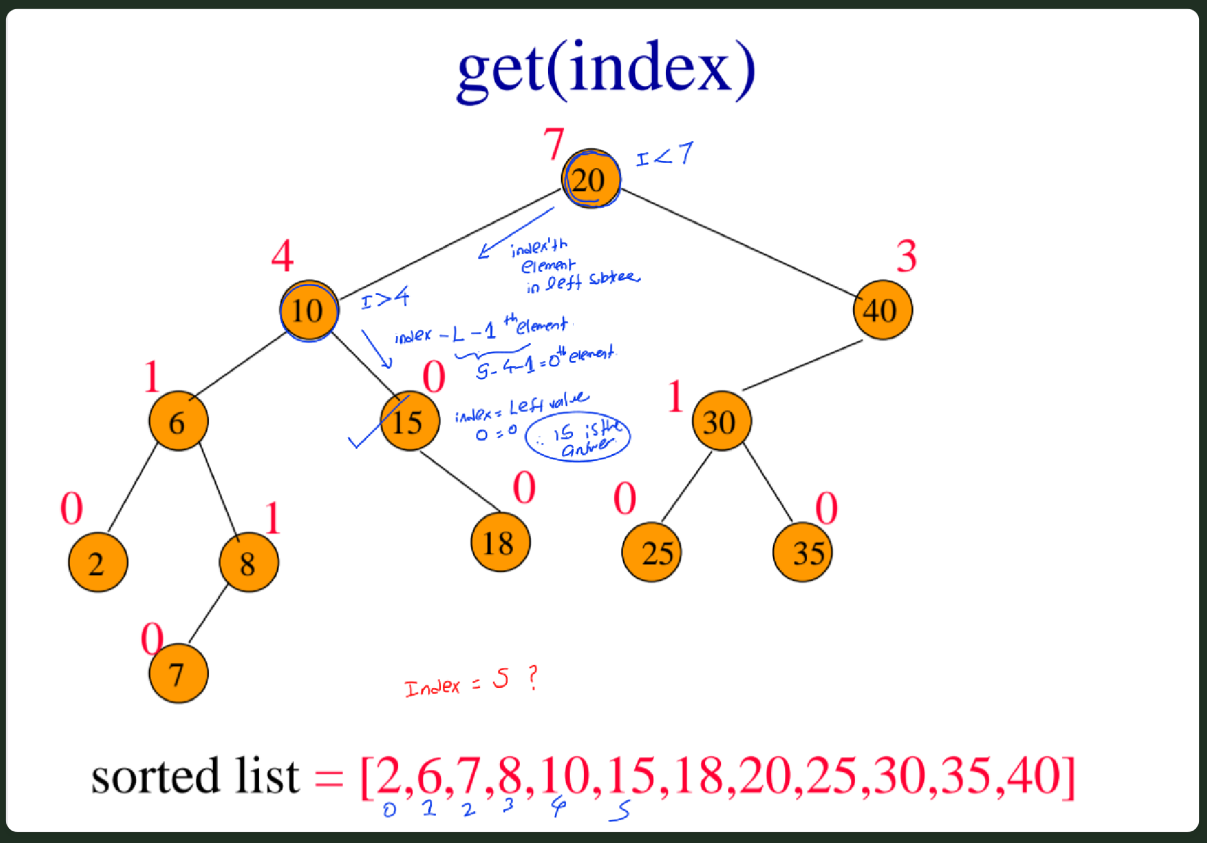
Condition	Action
$i = L$	Return the item at x
$i < L$	Go left, keeping index i
$i > L$	Go right with index $i - L - 1$

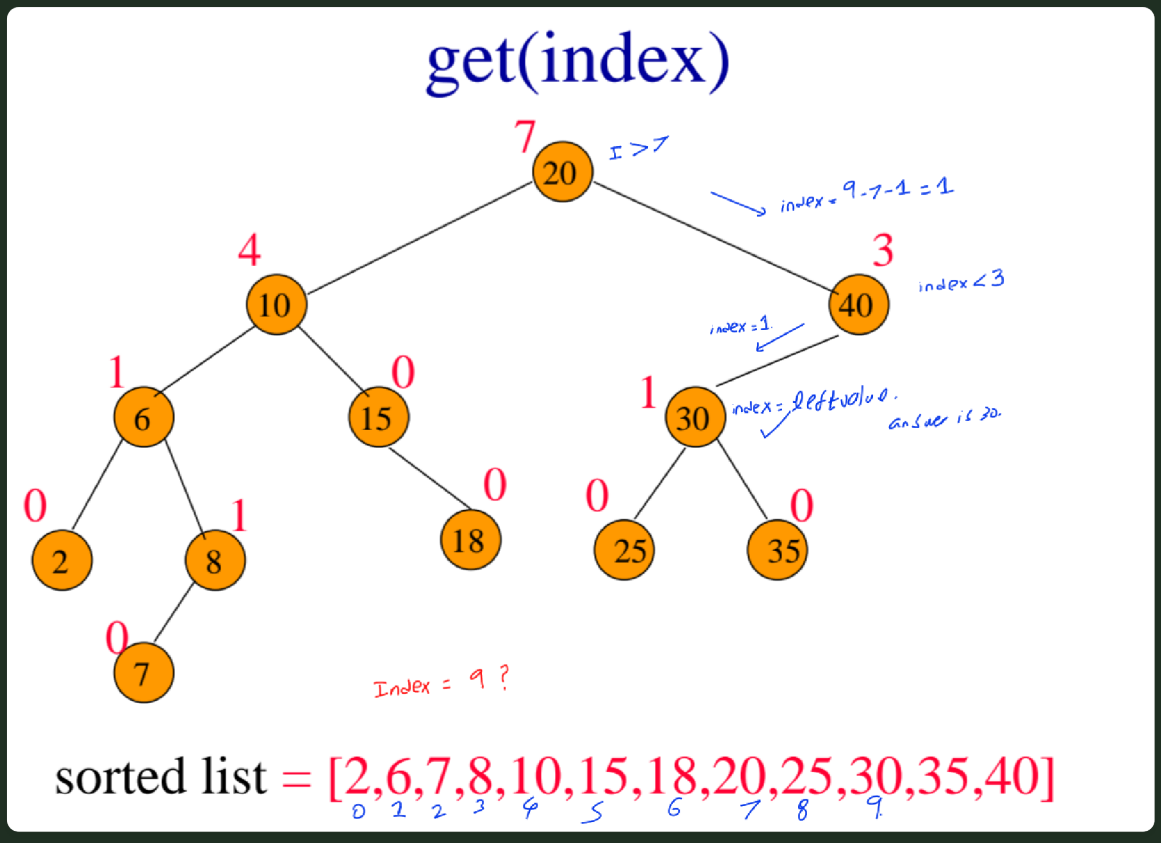
When going right, subtract L for all items in the left subtree and another 1 for the current node.

```python
getByIndex(root, index):
    reject index if it is outside 0 ... size-1
    x = root

    while x is not null:
        if index == x.leftSize:
            return x.element
        if index < x.leftSize:
            x = x.left
        else:
            index = index - x.leftSize - 1
```

```
            x = x.right
```





Example: `getByIndex(10)`

1. At 20, `leftSize = 7`. Go right with index $10 - 7 - 1 = 2$.

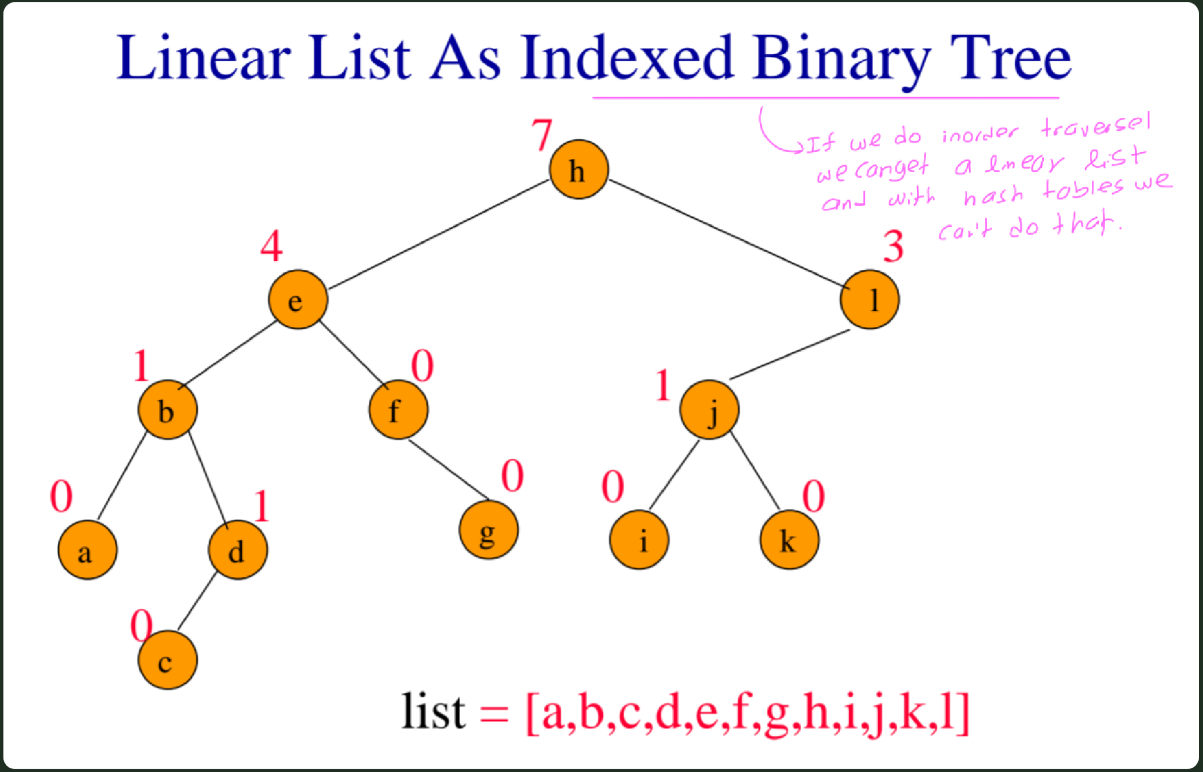
2. At 40, `leftSize = 3`. Go left, keeping index 2.
3. At 30, `leftSize = 1`. Go right with index $2 - 1 - 1 = 0$.
4. At 35, `leftSize = 0`. The index matches, so return 35.



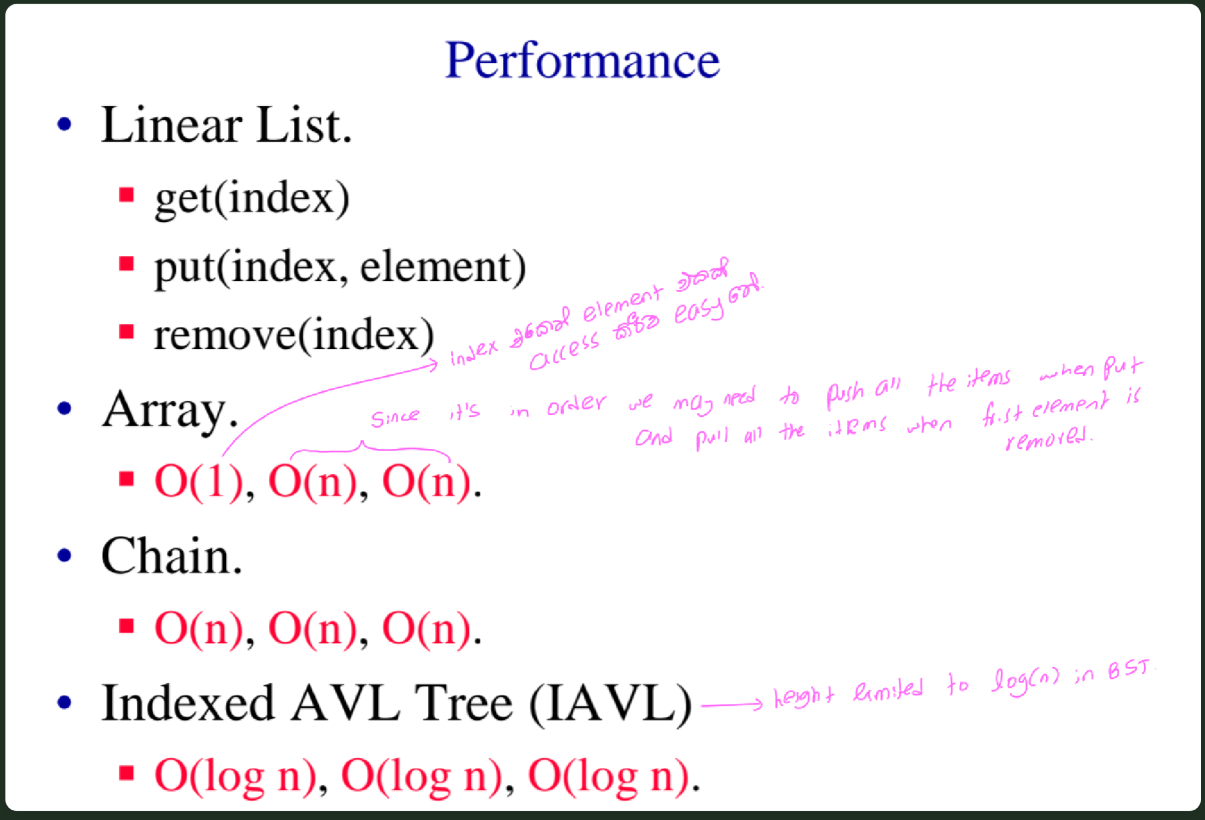
Only one downward path is followed, so the time is $O(h)$: $O(n)$ for a badly shaped tree and $O(\log n)$ for a balanced one.

Representing a Linear List

An indexed binary tree can also represent a list: its inorder sequence is the list's order. Here, positions determine the order; the stored values themselves do not have to be sorted.



List representation	`get(index)`	Insert at index	Remove at index
Array	$O(1)$	$O(n)$	$O(n)$
Linked chain	$O(n)$	$O(n)$	$O(n)$
Indexed AVL tree	$O(\log n)$	$O(\log n)$	$O(\log n)$



The linked-list bounds include locating the requested index. Inserting next to an already known node can be constant time, but finding that node by index is linear.

An indexed balanced tree must maintain its size information during insertion, deletion, and rebalancing. When a node gains or loses an item in its left subtree, its `leftSize` changes; rotations also require local size updates.

Experiment in the lecture

The slides report 40,000 operations of each type using Java on a 350 MHz PC:

Performance

Indexed AVL Tree (IAVL)

Operation	Array	Chain	IAVL
get	5.6ms	157sec	63ms
average puts	5.8sec	115sec	392ms
worst-case puts	11.8sec	157sec	544ms
average removes	5.8sec	149sec	1.5sec
worst-case removes	11.7sec	157sec	1.6sec

Time for 40,000 operations

Arrays are fastest for direct indexed access in this experiment, while indexed AVL trees handle positional insertions and removals much more efficiently. These are historical measurements for the stated setup, not general timings for current hardware.

Therefore using tree structures is good. Hashtable sucks at nearest match queries, Range queries, indexed operations and getting sorted output

- Tree structures for static and dynamic dictionaries.

Static Dictionaries

* Fixed කොහාවත් initialize කරිය යුත්තේ ඇයේ මොනාද වෙන්නේ නෑ.

- Collection of items.
- Each item is a pair.
  - (key, element)
  - Pairs have different keys.
- Operations are:
  - initialize/create
  - get (search)

Hashing

- Perfect hashing (no collisions).
- Minimal perfect hashing (space = n).
- CHD algorithm.
- O(n) time to construct the perfect or minimal perfect hash function.
- O(1) search time.
- Bothelo, Belazzougui & Dietzfelbinger. Compress, hash, and displace. *17th European Symposium on Algorithms, 2009*.

Perfect Hashing or a Search Tree?

Search Tree is better for Dictionaries,

Hashing is also possible for Static dictionaries

When the key set is fixed, it can be used to construct a hash function specifically for those keys.

- **Perfect hashing:** no two stored keys collide.
- **Minimal perfect hashing:** the n stored keys map to exactly n positions.

The lecture cites CHD (Compress, Hash and Displace) as a construction with $O(n)$ construction time and $O(1)$ function evaluation under its hashing assumptions. Minimality concerns the output positions; it does not mean the hash-function description and records require no additional storage. CHD project documentation

An absent key may still map to an occupied position, so a dictionary must verify the stored key to recognize an unsuccessful lookup. *Perfect hashing also does not solve ordered queries such as nearest match or range search.*

For a static BST, suppose we know how frequently each key is requested. A frequently requested key benefits from being closer to the root.

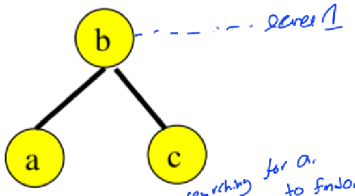
Example

Key	Probability
a	0.8
b	0.1
c	0.1

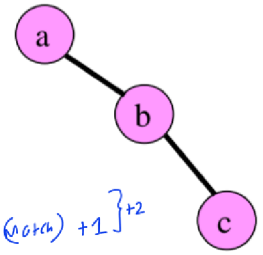
probability of searching 'a'

* If you know probability you can create a better tree.

costs of finding an element



a < b < c



when searching for a, to find out first need to compare with b +1 since a is less than b need to go to left subtree and compare. (match) +1 } +2

Cost = 0.8 * 2 + 0.1 * 1 + 0.1 * 2
= 1.9

when searching for b

when searching for c

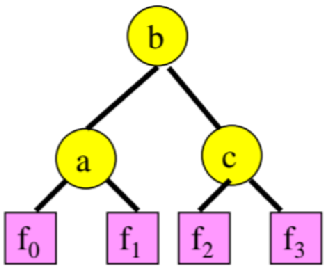
Cost = 0.8 * 1 + 0.1 * 2 + 0.1 * 3
= 1.3

33/77

The taller tree is cheaper on average because the most likely request finishes immediately. *An optimal BST minimizes this weighted search cost, rather than height alone.*

Successful and Unsuccessful Searches

Search Types



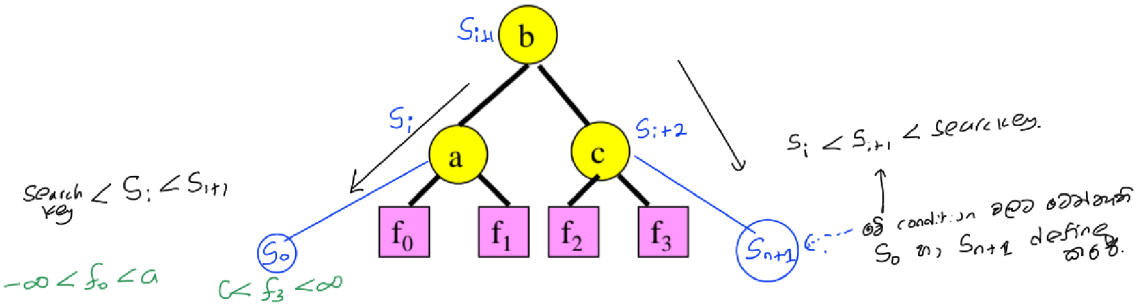
added failure nodes.

- Successful.
  - Search for a key that is in the dictionary.
  - Terminates at an internal node.
- Unsuccessful.
  - Search for a key that is not in the dictionary.
  - Terminates at an external/failure node.

In this extended-tree representation, the n key-containing nodes are called internal nodes, even if they have no real children. Replacing every missing child pointer with a failure node gives $n + 1$ external nodes:

$$f_0, f_1, \cdots, f_n$$

Internal And External Nodes



- A binary tree with n internal nodes has n + 1 external nodes.
- Let $s_1, s_2, \ldots, s_n$ be the internal nodes, in inorder.
- $key(s_1) < key(s_2) < \ldots < key(s_n)$.
- Let $key(s_0) = -infinity$ and $key(s_{n+1}) = infinity$.
- Let $f_0, f_1, \ldots, f_n$ be the external nodes, in inorder.
- f_i is reached iff $key(s_i) < search\ key < key(s_{i+1})$.

35/77

Each failure node corresponds to one gap:

- f_0: search key is less than a_1.
- f_i, for $1 \leq i < n$: $a_i <$ search key $< a_{i+1}$.
- f_n: search key is greater than a_n.

$(-\infty)\ s_0 < f_0 < s_1\ (a)$

$(a)\ s_1 < f_1 < s_2\ (b)$

$(b)\ s_2 < f_2 < s_3\ (c)$

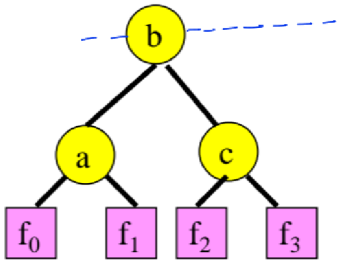
$(c)\ s_3 < f_3 < s_4\ (\infty)$

Let p_i be the probability of searching successfully for a_i, and q_i the probability of an unsuccessful search ending at f_i. Then:

$$\sum_{i=1}^{n} p_i + \sum_{i=0}^{n} q_i = 1$$

The Cost Formula

Cost Of Binary Search Tree



- Let p_i = probability for key(s_i).
- Let q_i = probability for key(s_i) < search key < key(s_{i+1}).
- Sum of ps and qs = 1.
- Cost of tree = $\Sigma_{0 <= i <= n} q_i$ (level(f_i) − 1) + $\Sigma_{1 <= i <= n} p_i$ * level(s_i)
- Cost = weighted path length.

The lecture numbers levels starting at **1 for the root**. Search cost counts comparisons with stored keys:

- A successful search at level ℓ makes ℓ key comparisons.
- A failed search reaching an external node at level ℓ makes $\ell - 1$ key comparisons, because the external node contains no key to compare.

Therefore:

$$C(T) = \sum_{i=1}^{n} p_i \,\text{level}(a_i) + \sum_{i=0}^{n} q_i \big(\text{level}(f_i) - 1\big)$$

For the balanced three-key tree above, every failure node is at level 3. Each failed search compares two real keys, so its contribution is $2q_i$.

An **optimal binary search tree** minimizes $C(T)$ over all BSTs for the given keys and probabilities. *Unlike a Huffman tree, it must preserve the key order*, and successful searches finish at key-containing nodes rather than message leaves.

* Our goal is to find a BST that minimizes the cost. One possible way is to try all possible BST's you can build.

Brute Force Algorithm

- Generate all binary search trees with n internal nodes.
- Compute the weighted path length of each.
- Determine tree with minimum weighted path length.
- Number of trees to examine is $O(4^n/n^{1.5})$. ← no need to worry about derivation
- Brute force approach is impractical for large n.

Brute force would enumerate all BST shapes and evaluate each cost. The number of shapes is the Catalan number:

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \Theta\left(\frac{4^n}{n^{3/2}}\right)$$

This grows exponentially, motivating dynamic programming.

5. Building an Optimal BST with Dynamic Programming

Subproblems and Base Cases

The lecture uses the following interval convention:

$$T_{ij} = \text{least-cost tree containing } a_{i+1}, \ldots, a_j$$

There are $j - i$ keys in this tree. The first index is the gap immediately before the first included key, so T_{ij} begins with a_{i+1}, not a_i.

For example, $T_{2,7}$ contains keys $a_3, \ldots, a_7$ and failure nodes $f_2, \ldots, f_7$:

Since brute force is impractical for finding best BST, we go with dynamic programming to find it

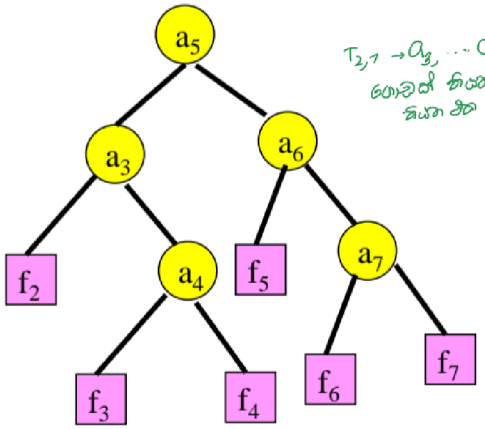
Dynamic Programming

- Keys are $a_1 < a_2 < \ldots < a_n$.
- Let T_{ij} = least cost tree for $a_{i+1}, a_{i+2}, \ldots, a_j$.

if you have nodes from i+1 to j (internal nodes) then least cost tree එක.

- T_{0n} = least cost tree for $a_1, a_2, \ldots, a_n$.

This is our final goal



$T_{2,7}$ → $a_3, \ldots a_7$ වලින් හදන්න පුළුවන් BST's ගොඩක් තියෙනවනේ. ඒ ඉන් අඩුම minimum cost තියන එක තමා අපට ඕන එක.

$T_{2,7}$
3 to 7

i+1 සිට j දක්වා

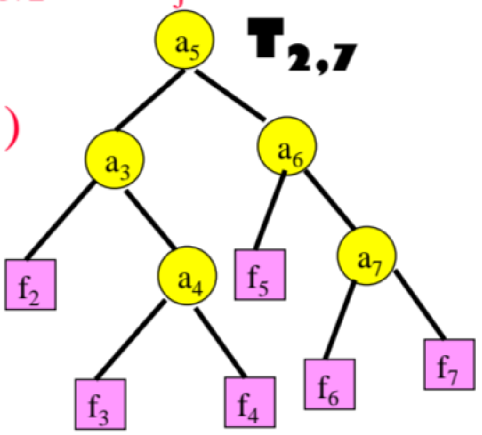
- T_{ij} includes $p_{i+1}, p_{i+2}, \ldots, p_j$ and $q_i, q_{i+1}, \ldots, q_j$.

internal නෝඩ් i+1 to j

external node ඒවා i to j

Terminology

- T_{ij} = least cost tree for $a_{i+1}, a_{i+2}, \ldots, a_j$.
- c_{ij} = cost of T_{ij}
  $= \Sigma_{i <= u <= j} q_u (level(f_u) - 1)$
  $+ \Sigma_{i < u <= j} p_u * level(s_u)$.
- r_{ij} = root of T_{ij}.
- w_{ij} = weight of T_{ij}
  = sum of ps and qs in T_{ij}
  $= p_{i+1} + p_{i+2} + \ldots + p_j + q_i + q_{i+1} + \ldots + q_j$

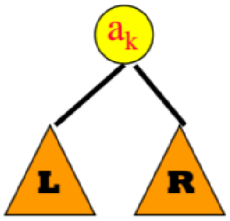


i = j

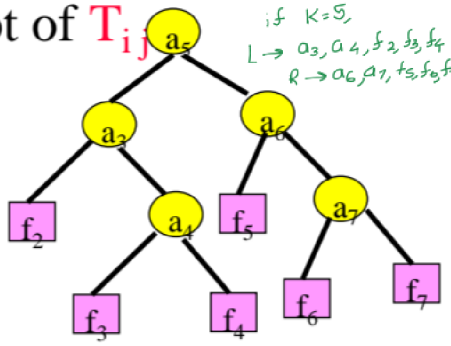
- $T_{i\,j}$ includes $p_{i+1}, p_{i+2}, \ldots, p_j$ and $q_i, q_{i+1}, \ldots, q_j$.
- $T_{i\,i}$ includes q_i only.
- $c_{i\,i}$ = cost of $T_{i\,i} = 0$.
- $r_{i\,i}$ = root of $T_{i\,i} = 0$.
- $w_{i\,i}$ = weight of $T_{i\,i}$
  = sum of ps and qs in $T_{i\,i}$
  = q_i

i < j

- $T_{i\,j}$ = least cost tree for $a_{i+1}, a_{i+2}, \ldots, a_j$.
- $T_{i\,j}$ includes $p_{i+1}, p_{i+2}, \ldots, p_j$ and $q_i, q_{i+1}, \ldots, q_j$.
- Let a_k, $i < k <= j$, be in the root of $T_{i\,j}$

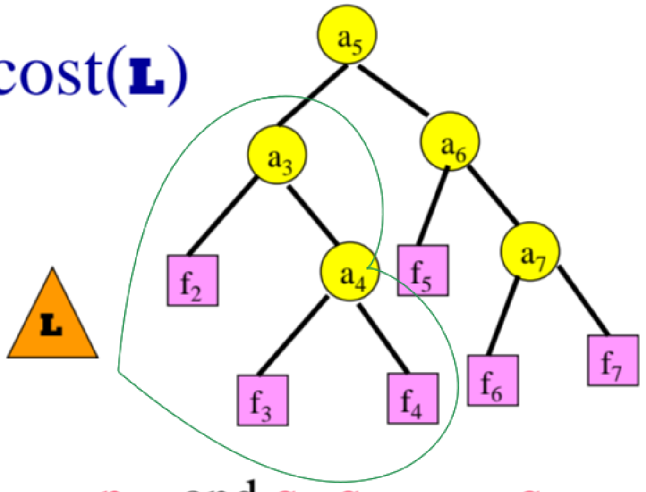


$T_{i\,j}$



- **L** includes $p_{i+1}, p_{i+2}, \ldots, p_{k-1}$ and $q_i, q_{i+1}, \ldots, q_{k-1}$.
- **R** includes $p_{k+1}, p_{k+2}, \ldots, p_j$ and $q_k, q_{k+1}, \ldots, q_j$.

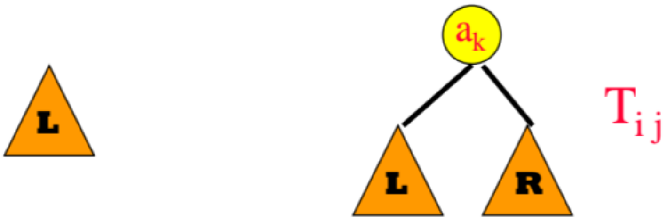
cost(L)



- **L** includes $p_{i+1}, p_{i+2}, \ldots, p_{k-1}$ and $q_i, q_{i+1}, \ldots, q_{k-1}$.
- cost(**L**) = weighted path length of **L** when viewed as a stand alone binary search tree.

$p_3 a_3 + p_4 a_4 + q_2 f_2 + q_3 f_3 + q_4 f_4$

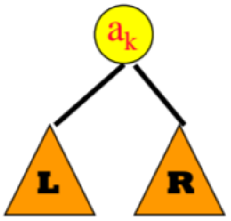
Contribution To c_{ij}

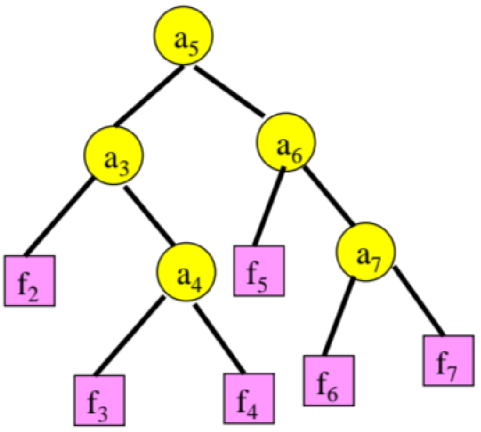


T_{ij}

- $c_{ij} = \Sigma_{i <= u <= j} q_u (\text{level}(f_u) - 1)$
  $+ \Sigma_{i < u <= j} p_u * \text{level}(s_u)$.
- When **L** is viewed as a subtree of T_{ij}, the level of each node is 1 more than when **L** is viewed as a stand alone tree.
- So, contribution of **L** to c_{ij} is cost(**L**) + $w_{i\,k-1}$.

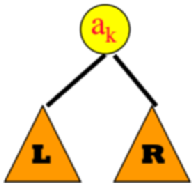
level x Probability
→ Right එකෙන් කෙළින් වෙන්න ගොඩාක් level එකේ වටි නම් root එක ඉතාමත් එකෙන් වල්ල.
left එකෙන් සහ implement

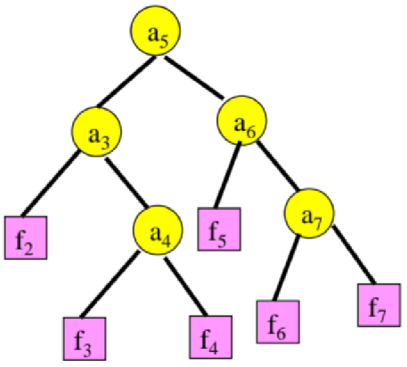




- Contribution of **L** to c_{ij} is cost(**L**) + $w_{i\,k-1}$.
- Contribution of **R** to c_{ij} is cost(**R**) + w_{kj}.
- c_{ij} = cost(**L**) + $w_{i\,k-1}$ + cost(**R**) + w_{kj} + p_k
  = cost(**L**) + cost(**R**) + w_{ij}

$w_{i,k-1} + w_{kj} + p_k = w_{ij}$

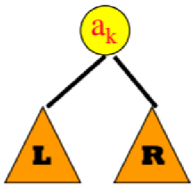




- c_{ij} = cost(**L**) + cost(**R**) + w_{ij}
- cost(**L**) = c_{ik-1}
- cost(**R**) = c_{kj}
- c_{ij} = c_{ik-1} + c_{kj} + w_{ij}
- Don't know k.
- $c_{ij} = \min_{i < k <= j}\{c_{ik-1} + c_{kj}\} + w_{ij}$

left right root + addition of a level weights.

c_{ij}



T_{ij}

- $c_{ij} = \min_{i < k <= j}\{c_{ik-1} + c_{kj}\} + w_{ij}$
- $r_{ij} = k$ that minimizes right side.

root ඒක

Computation Of c_{0n} And r_{0n}

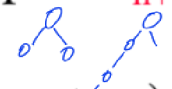
ex: C_{25} ගණනය කළා නම් එම සංඛ්‍යාවයට අදාල වූ cost එකක් එක්ක K අගයත් C_{25} min cost එක store කළ යුතුයි C_{26} ගණනය කිරීමේදී C_{25} කියන අගය අවශ්‍ය වෙනවා. pairwise.

- Start with $c_{i\,i} = 0$, $r_{i\,i} = 0$, $w_{i\,i} = q_i$, $0 <= i <= n$ (zero-key trees).
- Use $c_{ij} = \min_{i < k <= j}\{c_{ik-1} + c_{kj}\} + w_{ij}$ to compute c_{ii+1}, $r_{i\,i+1}$, $0 <= i <= n - 1$ (one-key trees). computing individual costs for n trees



no range to calculate. K is just i+1.

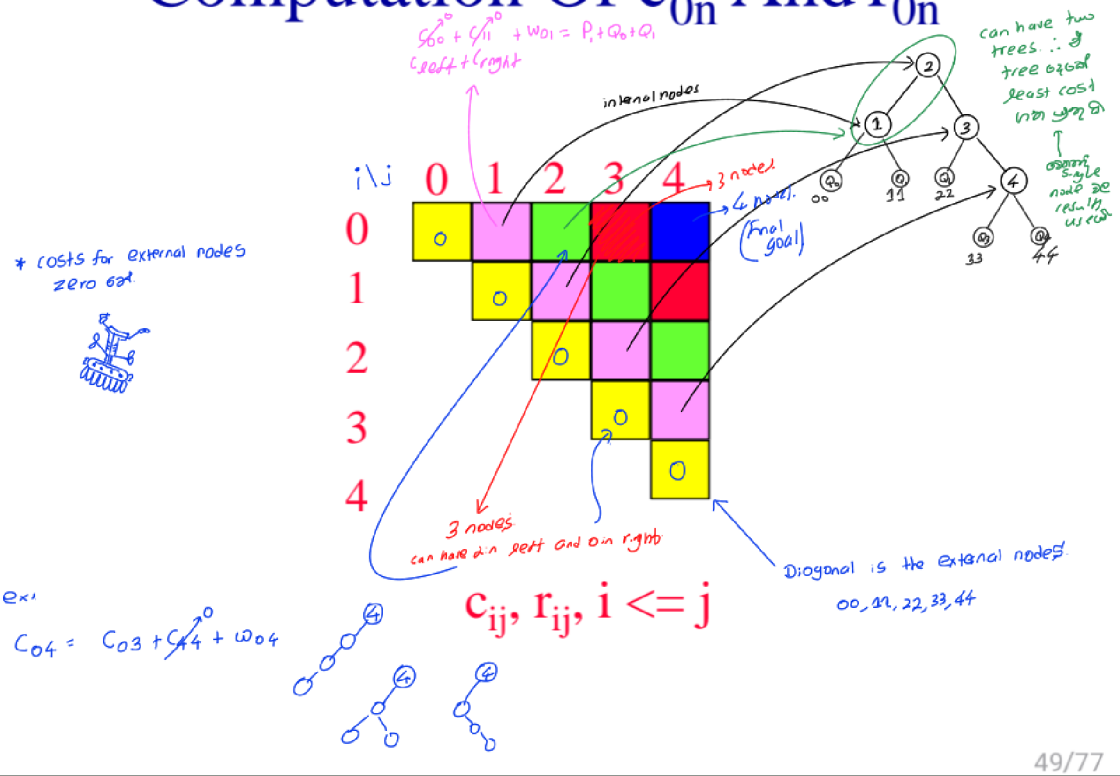
- Now use the equation to compute c_{ii+2}, $r_{i\,i+2}$, $0 <= i <= n - 2$ (two-key trees). $C_{02}, C_{13}, C_{24}, C_{35}, \ldots$ computing individual costs for two node trees → for this we need best of 1 node trees. so previous result is used.
- Now use the equation to compute c_{ii+3}, $r_{i\,i+3}$, $0 <= i <= n - 3$ (three-key trees). $C_{03}, C_{14}, C_{25}, \ldots$



we needs 1 and 0 nodes optimal results.

- Continue until c_{0n} and r_{0n} (n-key tree) have been computed.

Computation Of c_{0n} And r_{0n}



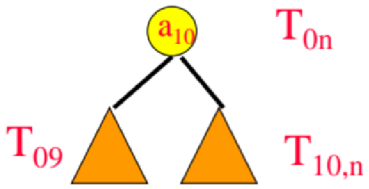
Complexity

- $c_{ij} = \min_{i < k <= j}\{c_{ik-1} + c_{kj}\} + w_{ij}$
- $O(n)$ time to compute one c_{ij}.
- $O(n^2)$ c_{ij}s to compute.
- Total time is $O(n^3)$.
- May be reduced to $O(n^2)$ by using

$$c_{ij} = \min_{r_{i,j-1} < k <= r_{i+1,j}}\{c_{ik-1} + c_{kj}\} + w_{ij}$$

Construct T_{0n}

- Root is r_{0n}.
- Suppose that $r_{0n} = 10$.



- Construct T_{09} and $T_{10,n}$ recursively.
- Time is O(n).

Symbol	Meaning
T_{ij}	Optimal tree for keys $a_{i+1}, \ldots, a_j$
c_{ij}	Cost of that tree, measured with its own root at level 1
r_{ij}	Index of the key chosen as its root
w_{ij}	Total probability of all successful and failed searches in this interval

The weight is:

$$w_{ij} = \sum_{u=i+1}^{j} p_u + \sum_{u=i}^{j} q_u$$

These are the original probabilities; do not renormalize them separately for each subtree.

When $i = j$, the interval contains no keys and only the failure node f_i:

$$c_{ii} = 0, \qquad r_{ii} = 0, \qquad w_{ii} = q_i$$

The cost is zero because there are no stored keys to compare. The weight is still q_i, because searches can reach this empty subtree. Here `r = 0` is a sentinel meaning no key-containing root.

Deriving the Recurrence

Suppose we choose a_k as the root of T_{ij}, where $i < k \leq j$.

The BST ordering rule determines the two subproblems:

$$\text{left subtree} = T_{i,k-1}, \qquad \text{right subtree} = T_{k,j}$$

The right subtree starts at a_{k+1}, which is why its interval is $T_{k,j}$.

When we attach a subtree below the new root, every search within it performs one extra comparison. Thus:

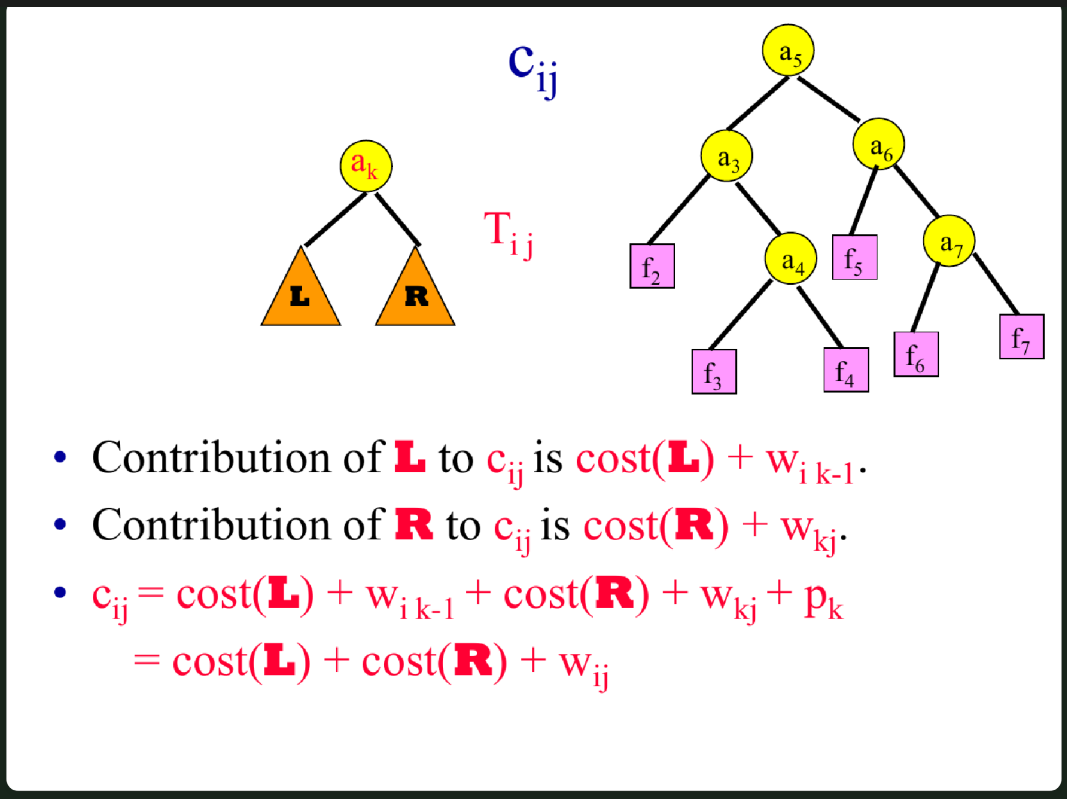
Part	Contribution to the complete tree's cost
Left subtree	$c_{i,k-1} + w_{i,k-1}$
Right subtree	$c_{k,j} + w_{k,j}$
Root key itself	p_k

The two subtree weights and the root probability together cover all searches in the interval:

$$w_{i,k-1} + w_{k,j} + p_k = w_{ij}$$

Therefore, choosing root a_k gives cost:

$$c_{i,k-1} + c_{k,j} + w_{ij}$$



- Contribution of **L** to c_{ij} is cost(**L**) + $w_{i\,k-1}$.
- Contribution of **R** to c_{ij} is cost(**R**) + w_{kj}.
- c_{ij} = cost(**L**) + $w_{i\,k-1}$ + cost(**R**) + w_{kj} + p_k
  = cost(**L**) + cost(**R**) + w_{ij}

We do not know which root is best, so try every allowed k:

$$\boxed{c_{ij} = w_{ij} + \min_{i<k\le j} (c_{i,k-1} + c_{k,j})}$$

Store the minimizing k in r_{ij}.

> **Why add w_{ij}?** Every search represented by this interval compares against its new root once. Multiplying that extra comparison by the probability of each possible search, then adding, gives exactly w_{ij}.

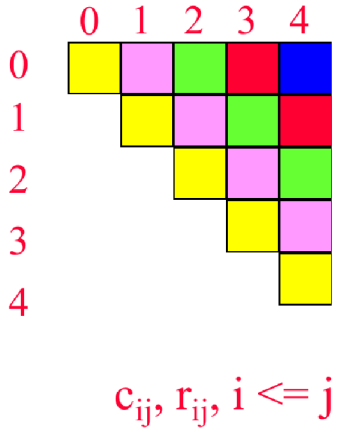
This recurrence uses **optimal substructure**: once a root is fixed, both subtrees must be optimal for their intervals. Replacing an inferior subtree with a cheaper one would improve the entire tree without changing BST order.

Filling the Tables and Reconstructing the Tree

Compute intervals in increasing order of their number of keys:

```text
0-key intervals: c[i,i]
1-key intervals: c[i,i+1]
2-key intervals: c[i,i+2]
...
n-key interval:  c[0,n]
```

Computation Of c_{0n} And r_{0n}



c_{ij}, r_{ij}, $i <= j$

The weights can also be built incrementally:

$$w_{ij} = w_{i,j-1} + p_j + q_j$$

```text
for i = 0 to n:
    c[i,i] = 0
    r[i,i] = 0
    w[i,i] = q[i]

for length = 1 to n:
    for i = 0 to n-length:
        j = i + length
        w[i,j] = w[i,j-1] + p[j] + q[j]
```

```
            c[i,j] = infinity

            for k = i+1 to j:
                candidate = c[i,k-1] + c[k,j] +
    w[i,j]
                if candidate < c[i,j]:
                    c[i,j] = candidate
                    r[i,j] = k
```

There are $O(n^2)$ intervals and up to $O(n)$ root candidates per interval:

$$\text{time} = O(n^3), \qquad \text{space} = O(n^2)$$

Once the root table is complete, build the tree recursively:

text

```
build(i, j):
    if i == j:
        return null        // conceptually the
failure node f[i]

    k = r[i,j]
    x = node containing the item with key a[k]
    x.left = build(i, k-1)
    x.right = build(k, j)
    return x
```

Call `build(0, n)`. Each key becomes one node, so reconstruction takes $O(n)$ time. It does not repeat the search over root choices.

The $O(n^2)$ improvement

Knuth's root-monotonicity property, with consistent tie-breaking between equal-cost roots, gives:

$$r_{i,j-1} \le r_{ij} \le r_{i+1,j}$$

After initializing one-key intervals, longer intervals need only test roots between these two bounds, inclusive. This reduces the total number of candidate tests to $O(n^2)$; it does not mean every individual interval tests only a constant number of roots. Optimal-BST notes, University of Texas at Arlington

> The optimized formula on slide 48 prints a strict lower inequality. The lower endpoint must be included: $r_{i,j-1} \le k \le r_{i+1,j}$. Otherwise, a valid optimal root can be skipped.

Small Worked Example

Consider two keys $a < b$ with:

$$p_1 = 0.30, \quad p_2 = 0.20, \quad q_0 = 0.10, \quad q_1 = 0.15, \quad q_2 = 0.25$$

The probabilities sum to 1. Here q_0 represents searches below a, q_1 searches between a and b, and q_2 searches above b.

Empty intervals:

$$c_{00} = c_{11} = c_{22} = 0$$

One-key intervals: only one choice of root is possible.

$$c_{01} = w_{01} = q_0 + p_1 + q_1 = 0.55$$

$$c_{12} = w_{12} = q_1 + p_2 + q_2 = 0.60$$

Two-key interval: $w_{02} = 1$.

Root choice	Left cost	Right cost	Total
a $(k = 1)$	$c_{00} = 0$	$c_{12} = 0.60$	$0 + 0.60 + 1 = 1.60$
b $(k = 2)$	$c_{01} = 0.55$	$c_{22} = 0$	$0.55 + 0 + 1 = 1.55$

Therefore:

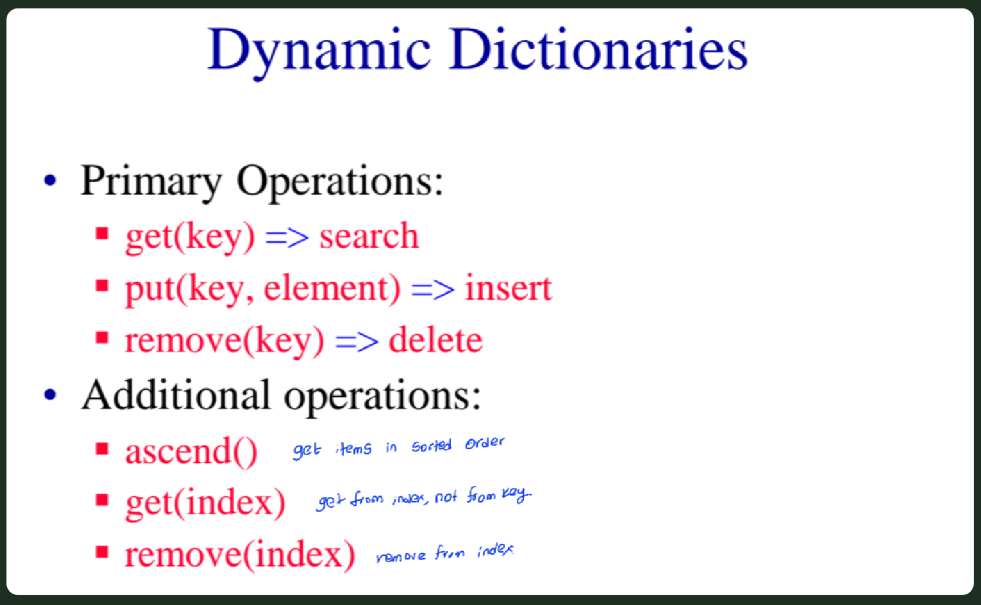
$$c_{02} = 1.55, \qquad r_{02} = 2$$

The optimal tree has b at the root and a as its left child. Its expected comparisons can be checked directly:

$$0.30(2) + 0.20(1) + 0.10(2) + 0.15(2) + 0.25(1) = 1.55$$

Even though a is the more frequently requested stored key, the failed-search probabilities make b the better root.

6. Dynamic BST Operations



A dynamic dictionary supports search, insertion, and deletion as its collection changes. Search trees can additionally support ascending traversal and, when augmented with size information, indexed access and deletion.

Complexity Of Dictionary Operations
get(), put() and remove()

Data Structure	Worst Case	Expected
Hash Table	O(n)	O(1)
Binary Search Tree	O(n)	O(log n)
Balanced Binary Search Tree	O(log n)	O(log n)

n is number of elements in dictionary

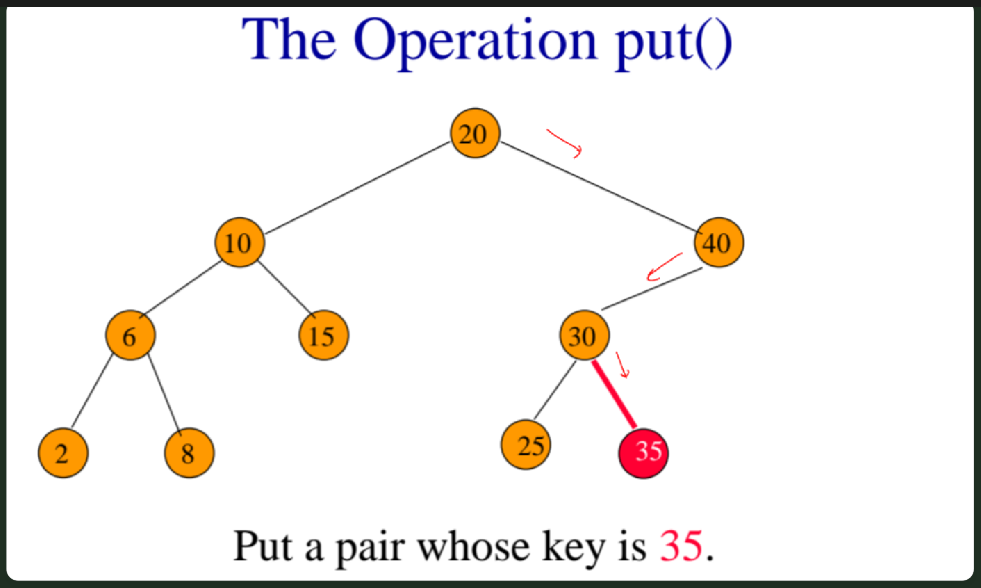
Complexity Of Other Operations
ascend(), get(index), remove(index)

Data Structure	ascend	get and remove by index
Hash Table	O(D + n log n)	O(D + n log n)
Indexed BST	O(n)	O(n)
Indexed Balanced BST	O(n)	O(log n)

D is number of buckets

Search and Insertion

For search, compare the requested key with the current node and follow the appropriate child. Insertion follows the same path until reaching a missing child position, where the new node is attached.

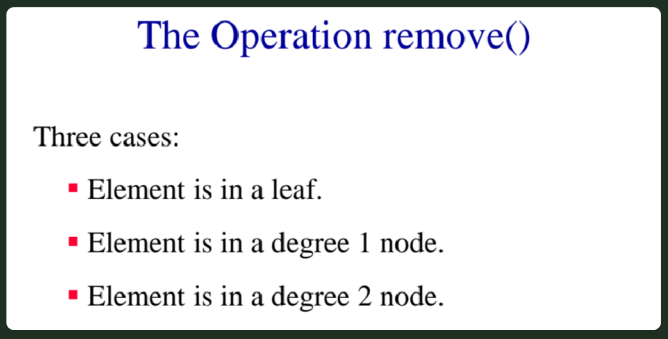


For example, inserting key 35 into the lecture's tree before 35 is present follows:

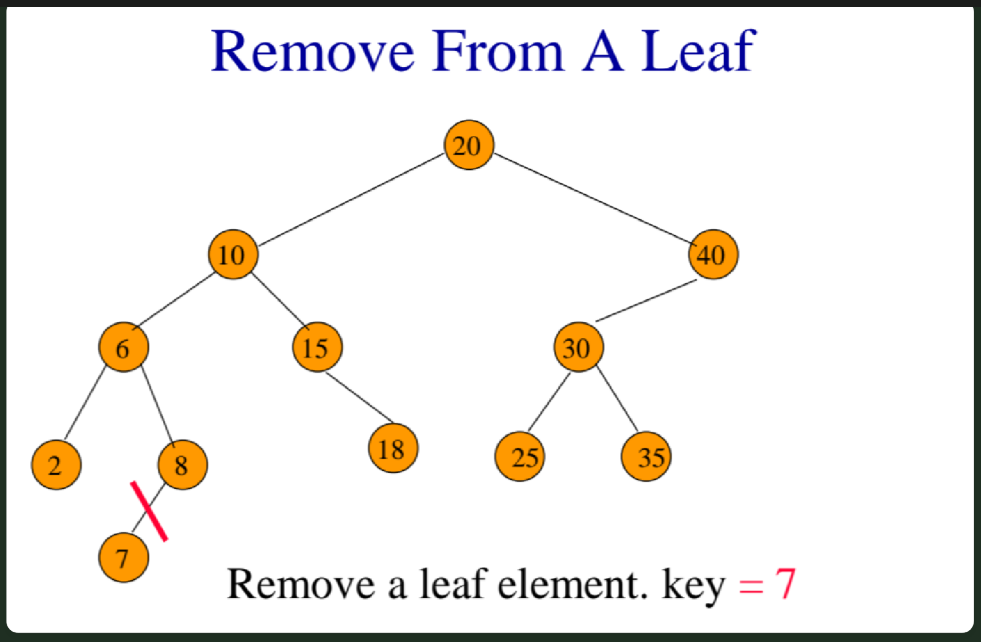
```text
20 -> 40 -> 30 -> missing right child
```

Both operations take $O(h)$ time in a BST of height h.

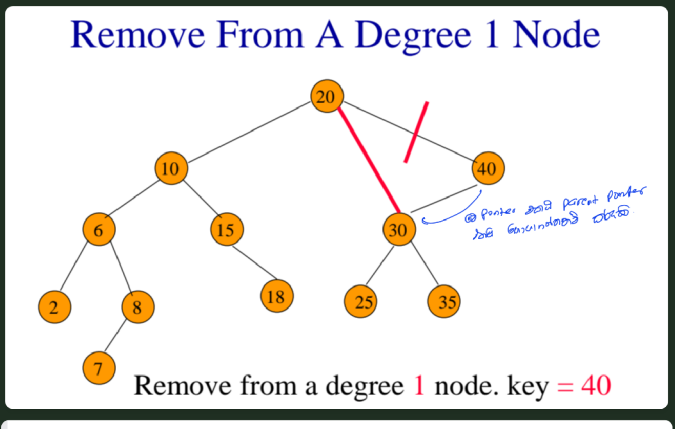
Deletion: Three Cases

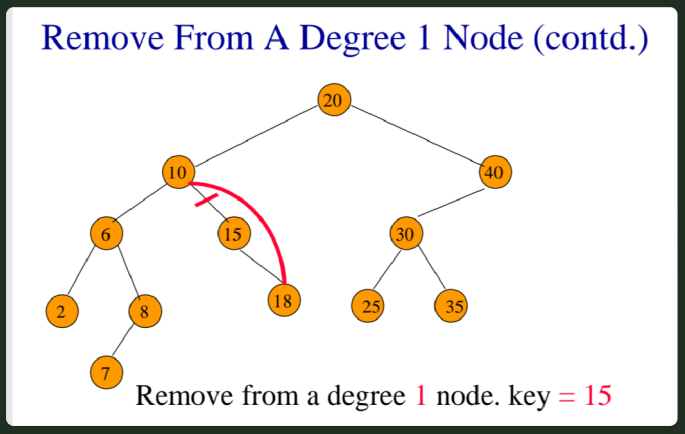


1. The node is a leaf. Remove it and set the corresponding parent pointer to `null`. In the lecture tree, key 7 is a leaf.



2. The node has one child. Link its parent directly to its child. If deleting the root, update the tree's root pointer instead. Examples include replacing 40 with its child 30, or replacing 15 with its child 18.

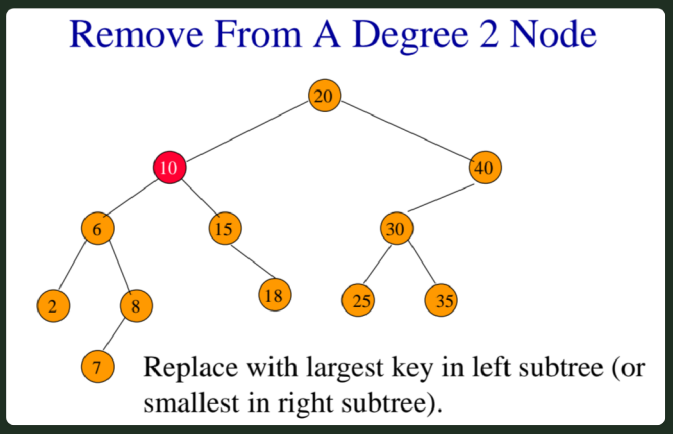


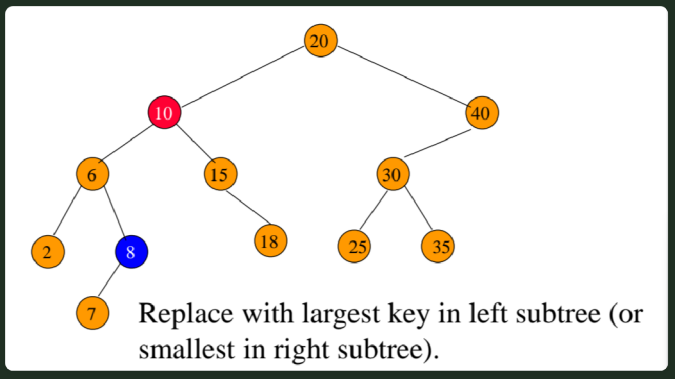


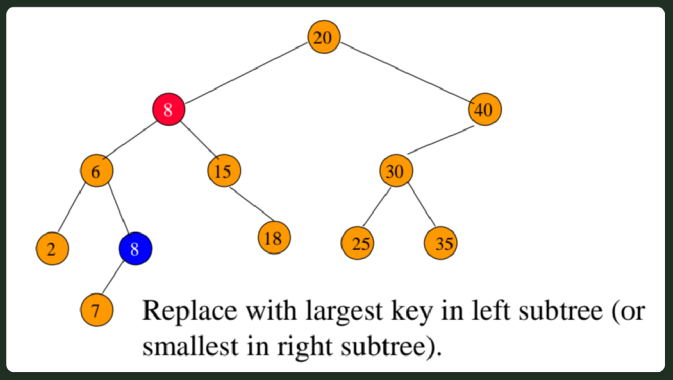
3. The node has two children. Replace its key and associated element with either:

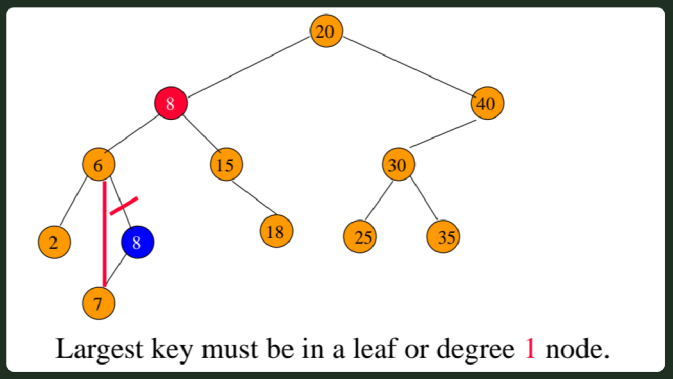
- the **inorder predecessor**, which is the largest key in its left subtree; or
- the **inorder successor**, which is the smallest key in its right subtree.

Then delete the predecessor/successor from its old location. That node has at most one child, so the remaining deletion is one of the simpler cases.

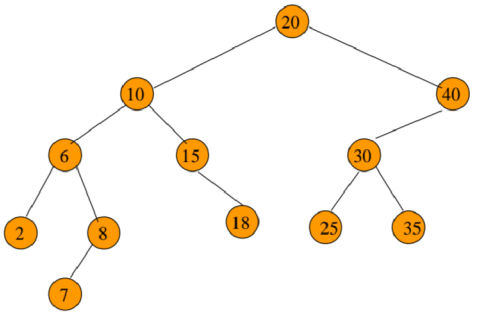




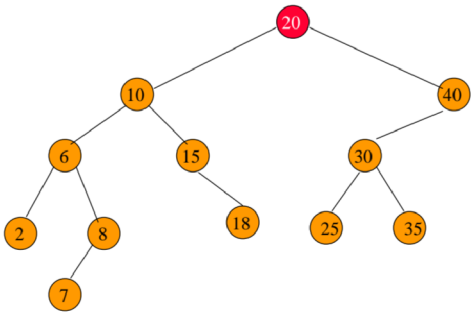




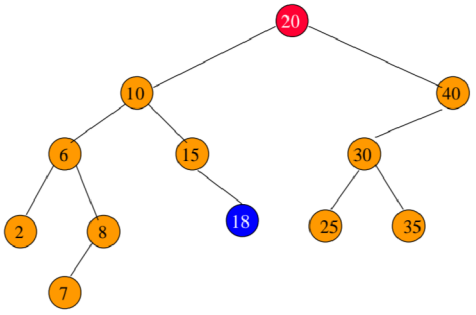
Another Remove From A Degree 2 Node



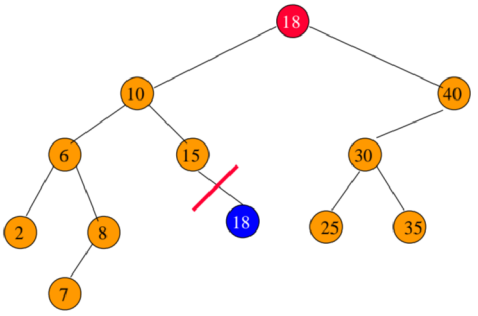
Remove from a degree 2 node. key = 20



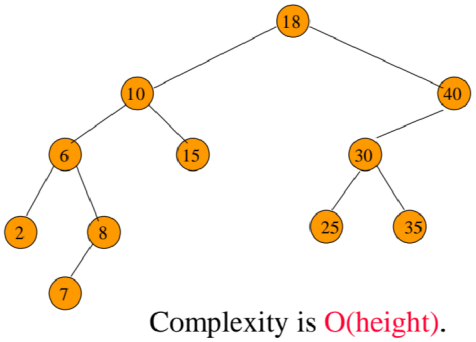
Replace with largest in left subtree.



Replace with largest in left subtree.



Replace with largest in left subtree.



Complexity is O(height).

Searching for the target and finding its replacement follows tree paths, so deletion takes $O(h)$. A balanced tree also restores its balance; an indexed tree must update the affected size fields.

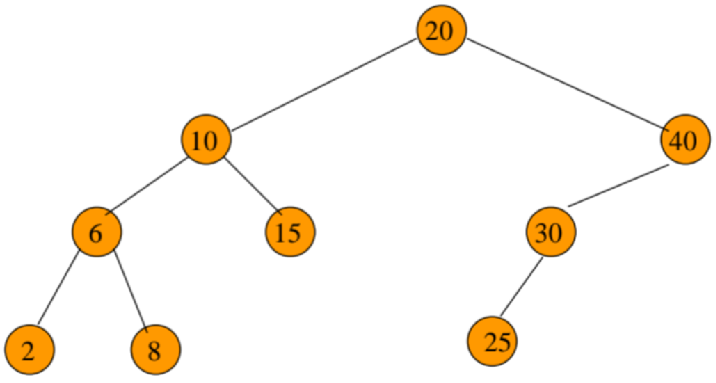
Yet Other Operations

- Priority Queue Motivated Operations:
  - find max and/or min leftmost leaf
  - remove max and/or min rightmost leaf
  - initialize
  - meld

7. Priority-Queue Operations and the Need for Balance

Minimum and maximum

Max And/Or Min



- Follow rightmost path to max element.
- Follow leftmost path to min element.
- Search and/or remove => O(h) time.

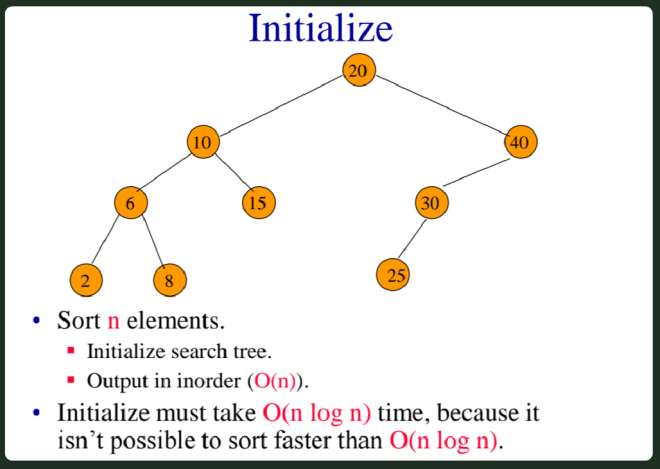
O(log n) balance BST

71/77

In a BST, the minimum is reached by following left children, while the maximum is reached by following right children. Finding or

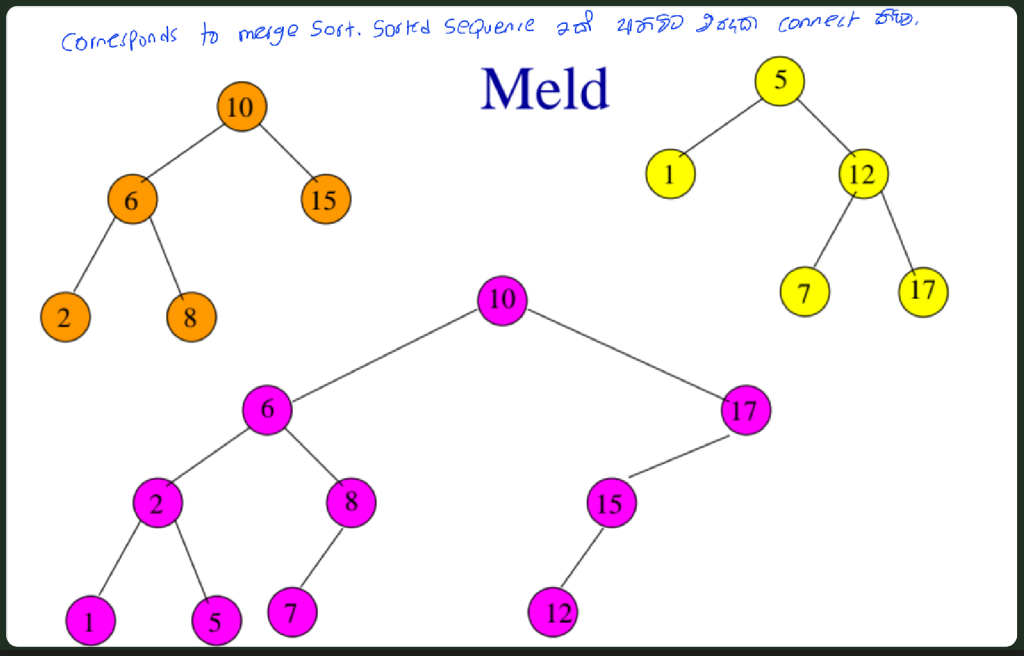
removing either extreme takes $O(h)$ without additional cached information.
Unlike a min heap, a BST does not necessarily store its minimum at the root. Its stronger ordering supports searches throughout the key range.

Initialization from unsorted input



Constructing a BST containing arbitrary comparison-based keys requires $\Omega(n \log n)$ comparisons in the worst case. Otherwise, constructing the tree and then performing an $O(n)$ inorder traversal would sort arbitrary inputs faster than the comparison-sorting lower bound.
A matching construction sorts the keys and recursively chooses middle keys as roots, taking $O(n \log n)$ overall. If the input is already sorted, the balanced tree can be built in $O(n)$.

Melding two BSTs



Meld And Merge

- Worst-case number of comparisons to merge two sorted lists of size n each is 2n-1.
- So, complexity of melding two binary search trees of size n each is Ω(n).
- So, logarithmic time melding isn't possible.

The keys from two BSTs may be arbitrarily interleaved. A general meld can:

1. traverse both trees in order;
2. merge the two sorted sequences;
3. build a balanced BST from the result.

For trees with m and n items, this takes $O(m+n)$ time. Merging two sorted lists of size n each may need $2n-1$ comparisons, and general comparison-based BST melding has a linear worst-case lower bound. Thus it cannot promise the constant or logarithmic meld cost seen in the specialized priority queues from earlier lectures.
This concerns unrestricted meld. Joining trees whose key ranges are already known to be disjoint and ordered is a different operation.

Why Not Force a Complete Tree?

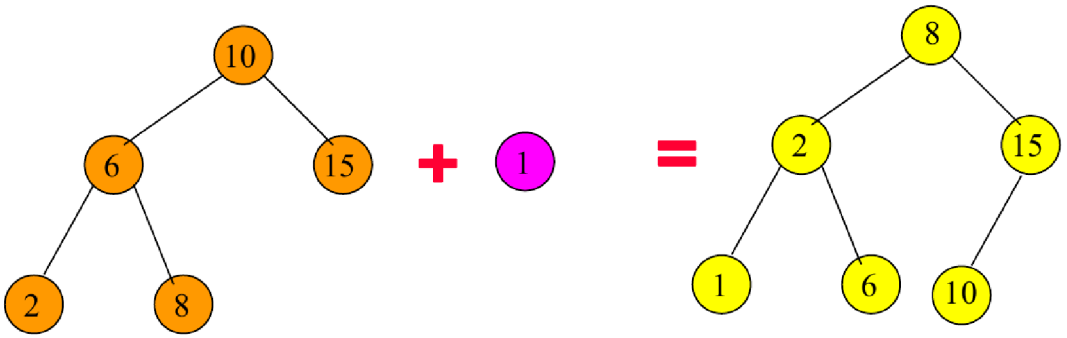
A perfect binary tree has all levels filled and can only have:

$$n = 2^k - 1$$

nodes for some number of levels k. The lecture calls these trees "full"; "perfect" makes the all-levels-filled condition explicit.
A complete binary tree can exist for every n, but maintaining both its exact complete shape and BST order can require many nodes to move when one key is inserted or deleted.

O(log n) Height Trees

- Full binary trees.
  - Exist only when $n = 2^k - 1$.
- Complete binary trees.
  - Exist for all n.
  - Cannot insert/delete in O(log n) time.



Balanced search trees use more flexible structural rules that preserve $O(\log n)$ height while allowing $O(\log n)$ updates. The lecture introduces these families:

Balancing approach	Idea or examples
Height balance	Restrict subtree-height differences; AVL trees
Weight balance	Restrict imbalance in subtree sizes
Degree balance	Control multiway node sizes; 2-3 trees, 2-3-4 trees, B-trees; red-black trees encode related balancing constraints in binary form

8. Quick Revision

Need	Structure or technique from this lecture
Expected constant-time exact-key operations	Hash table
Collision-free lookup for a fixed key set	Perfect hashing
Smallest key at least a threshold	BST lower-bound search
Best-fit bin packing in $O(n \log n)$	Balanced dictionary keyed by available capacity
Access an item by sorted position	Indexed BST with subtree counts
Logarithmic positional list updates	Indexed balanced tree
Minimize expected static search cost	Optimal BST using access probabilities
Guarantee logarithmic dynamic dictionary operations	Balanced search tree

Formulas to remember

For indexed lookup, go right with:

$$\text{new index} = \text{index} - \text{leftSize} - 1$$

For an optimal BST with the root at level 1:

$$C(T) = \sum_{i=1}^{n} p_i \,\text{level}(a_i) + \sum_{i=0}^{n} q_i \Big(\text{level}(f_i) - 1\Big)$$

$$c_{ii} = 0, \qquad w_{ii} = q_i$$

$$c_{ij} = w_{ij} + \min_{i<k\le j} (c_{i,k-1} + c_{k,j})$$

Common mistakes

- Treating `leftSize` as the node's global rank.
- Forgetting to subtract 1 for the current node when going right.
- Including a_i in T_{ij}; the first included key is a_{i+1}.
- Setting $w_{ii} = 0$ even though the empty interval still has failure probability q_i.
- Counting an extra key comparison at an external failure node.
- Assuming the most balanced tree always has the lowest expected search cost.
- Changing a dictionary key in place without restoring BST order.
- Assuming every ordinary BST has logarithmic height.

9. References

- Lecture 08 slides, CS4580 - Advanced Data Structures, slides 1-75.
- E. Horowitz, S. Sahni, and D. Mehta, *Fundamentals of Data Structures in C++*, 2nd ed., Silicon Press, 2007.
- Lecture slides of Prof. Sartaj Sahni.
- The CHD project documentation and University of Texas at Arlington optimal-BST notes are linked at the relevant explanations above.